NORMALIZATION

NORMALIZATION

- **Normalization**: The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations

- Normalization theory is based on the concepts of **normal forms**

- **Normal form**: Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

NORMALIZATION

- **Functional Dependencies**

A column, **Y**, of the relational table **R** is said to be **functionally dependent** upon column **X** of **R** if and only if each value of **X** in **R** is associated with precisely one value of **Y** at any given time. **X** and **Y** may be composite. Saying that column **Y** is functionally dependent upon **X** is the same as saying the values of column **X** identify the values of column **Y**. If column **X** is a primary key, then all columns in the relational table **R** must be functionally dependent upon **X**. A short-hand notation for describing a functional dependency is:

R.x —>; R.y

NORMALIZATION

- **First Normal Form**

  Test : Relation should have no nonatmoic attributes or nested relations

  Remedy: Form new relations for each nonatomic attribute or nested relation.

NORMALIZATION

FIRST

s#	status	city	p#	qty
s1	20	London	p1	300
s1	20	London	p2	200
s1	20	London	p3	400
s1	20	London	p4	200
s1	20	London	p5	100
s1	20	London	p6	100
s2	10	Paris	p1	300
s2	10	Paris	p2	400
s3	10	Paris	p2	200
s4	20	London	p2	200
s4	20	London	p4	300
s4	20	London	p5	400

NORMALIZATION

- **Second Normal Form (2NF)**

  Test: For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key

NORMALIZATION

- **Second Normal Form (2NF)**

  Remedy: Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.

NORMALIZATION

- **General Definition of 2NF**

  A relation schema **R** is in **2NF** if every nonprime attribute **A** in **R** is not partially dependent on any key of **R.**

NORMALIZATION

- **The process for transforming a 1NF table to 2NF is:**
  - Identify any determinants other than composite key, and the column they determines.
  - Create and name a new table for each determinant and the unique column it determines.

NORMALIZATION

- **Contd…**
  - Move the determined columns from the original table to the new table. The determinate becomes the primary key of the new table
  - Delete the columns you just moved from the original table except for the determinate which serve as a foreign key
  - The original table may be renamed to maintain semantic meaning.

NORMALIZATION

SECOND

s#	status	city
s1	20	London
s2	10	Paris
s3	10	Paris
s4	20	London
s5	30	Athens

PARTS

s#	p#	qty
s1	p1	300
s1	p2	200
s1	p3	400
s1	p4	200
s1	p5	100
s1	p6	100
s2	p1	300
s2	p2	400
s3	p2	200
s4	p2	200
s4	p4	300
s4	p5	400

NORMALIZATION

- **Third Normal Form (3NF)**

  Test: Relation should not have a nonkey attribute functionally determined by another nonkey attribute (or by a set of nonkey attributes.) That is, there should be no transitive dependency of a nonkey attribute on the primary key.

NORMALIZATION

- **Third Normal Form (3NF)**

  Remedy: Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s)

NORMALIZATION

- **General definition of 3NF**

  A relation schema **R** is in **3NF** if, whenever a nontrivial functional dependency **X → A** holds in **R**, either

  (a) **X** is superkey of **R**,

  or

  (b) **A** is prime attribute of **R**

NORMALIZATION

- **The process of transforming a table into 3NF is:**
  - Identify any determinants, other the primary key, and the columns they determines
  - Create and name a new table for each determinant and the unique columns it determines.

NORMALIZATION

- **Contd…**
  - Move the determined columns from the original table to the new table. The determinate becomes the primary key of the new table
  - Delete the columns you just moved from the original table except for the determinate which will serve as a foreign key.
  - The original table may be renamed to maintain sematic meaning.

NORMALIZATION

SUPPLIER_CITY

s#	city
s1	London
s2	Paris
s3	Paris
s4	London
s5	Athens

CITY_STATUS

city	status
London	20
Paris	10
Athens	30
Rome	50

NORMALIZATION

- **Advantages of 3NF**

  The advantages of having relational tables in 3NF is that it eliminates redundant data which in turn saves space and reduces manipulation anomalies.

NORMALIZATION

- **Table represented in “psuedo-SQL”**

  PARTS (#s, p#, qty)
  Primary key (s#, p#)
  Foreign key(s#) references SUPPLIER_CITY.s#

  SUPPLIER_CITY (s#, city)
  Primary key (s#)
  Foreign key (city) references CITY_STATUS.city

  CITY_STATUS (city, status)
  Primary key (city)

NORMALIZATION

- **Boyce – Codd Normal Form (BCNF)**

  BCNF is a more rigorous version of the 3NF deal with relational tables that had

  (a) composite candidate keys,

  (b) multiple candidate keys

NORMALIZATION

- **General definition of BCNF**

  A relation schema **R** is in **BCNF** if whenever a nontrivial functional dependency **X → A** holds in **R**, then **X** is a superkey of **R**.

  The only difference between the definition of **BCNF** and **3NF** is the condition (b) of 3NF, which allows **A** to be prime, is absent from BCNF.

NORMALIZATION

- **NF^2**: non-first normal form
- **1NF**: R is in 1NF. **iff** all domain values are atomic
- **2NF**: R is in 2. NF. **iff** R is in 1NF and every nonkey attribute is fully dependent on the key
- **3NF**: R is in 3NF **iff** R is 2NF and every nonkey attribute is non-transitively dependent on the key
- **BCNF**: R is in BCNF **iff** every determinant is a candidate key

- **Determinant**: an attribute on which some other attribute is fully functionally dependent.